

Программирование корпоративных систем Практическое занятие №2-3

Название проекта: Система централизованного логирования для распределённых приложений (Distributed Logging System — DLS)

Студент: Максимов Святослав Эдуардович **Группа:** ЭФБО-0Х-24 **Преподаватель:** Сокунов Дмитрий Антонович **Дата:** 24 мая 2026 г.

1. Сбор требований

1.1 Бизнес-требования

- BR-1: Повысить наблюдаемость распределённых приложений.
- BR-2: Сократить время обнаружения и диагностики ошибок.
- BR-3: Создать единое хранилище всех логов компании.
- BR-4: Обеспечить надёжное хранение логов без потерь.
- BR-5: Упростить работу разработчиков и DevOps-инженеров.

1.2 Пользовательские требования

- US-1: Как разработчик, я хочу отправлять логи из приложения по сети.
- US-2: Как DevOps-инженер, я хочу просматривать логи в реальном времени с фильтрами.
- US-3: Как администратор, я хочу автоматическую ротацию файлов логов по дате и размеру.
- US-4: Как DevOps, я хочу искать логи за любой период.
- US-5: Как администратор, я хочу настраивать уровни логирования.

1.3 Функциональные требования

- FR-1: Клиентская библиотека для отправки логов по TCP.
- FR-2: Сервер агрегации логов.
- FR-3: Запись логов в файлы с ротацией (по дате и размеру).
- FR-4: Просмотр и фильтрация логов в реальном времени.
- FR-5: Поддержка уровней: Debug, Info, Warning, Error, Fatal.
- FR-6: Структурированный формат лога (JSON).
- FR-7: Аутентификация клиентов.
- FR-8: Возможность поиска по логам.

1.4 Нефункциональные требования

- NFR-1: Производительность — минимум 5000 логов в секунду.
- NFR-2: Гарантированная доставка (TCP).
- NFR-3: Кросс-платформенность (Linux + Windows).
- NFR-4: Безопасность (аутентификация).
- NFR-5: Масштабируемость.
- NFR-6: Реализация сервера на C++20.

2. Software Requirements Specification (SRS)

2.1 Введение

Цель: Разработка системы централизованного логирования для распределённых приложений.

Область действия: Система состоит из клиентской библиотеки и серверной части для сбора, хранения и анализа логов.

Определения:

- DLS — Distributed Logging System
- TCP — Transmission Control Protocol
- JSON — JavaScript Object Notation

2.2 Общее описание

Описание продукта: Клиент-серверная система, позволяющая приложениям отправлять логи по сети на центральный сервер.

Функции продукта:

- Сбор логов от множества клиентов
- Надёжное хранение с ротацией
- Реал-тайм просмотр и фильтрация
- Поиск по логам

Пользователи: Разработчики, DevOps/SRE, администраторы.

2.3 Детальные требования

Методология: Гибридный подход (Wiegiers + User Stories).

(Требования см. в разделе 1)

2.4 UML-диаграммы

Диаграмма вариантов использования: Акторы: Developer, DevOps Engineer, Administrator, Client Application. Прецеденты: Отправка лога, Просмотр логов, Фильтрация, Поиск, Ротация файлов, Настройка уровней.

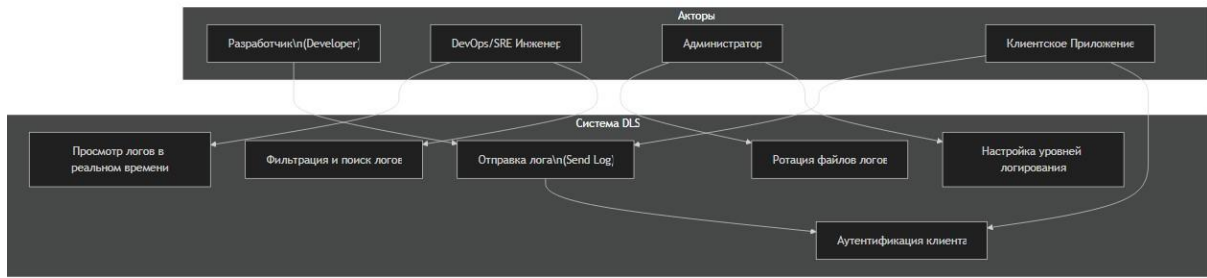


Диаграмма классов: Основные классы: LogMessage, LoggerClient, LogServer, LogStorage, FileRotator, LogFilter.

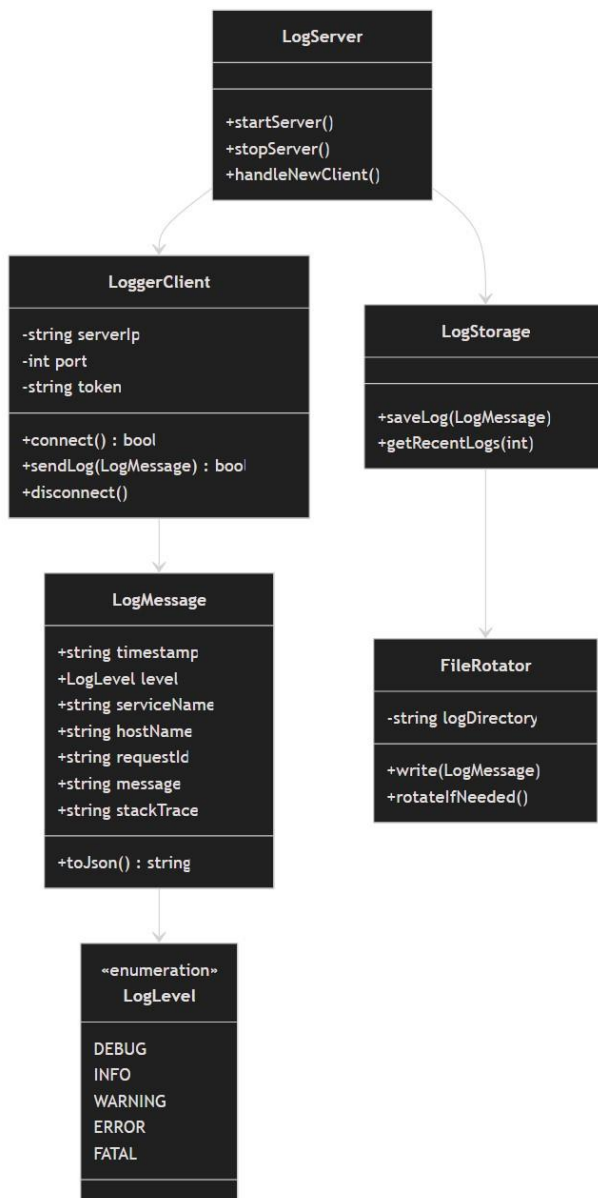
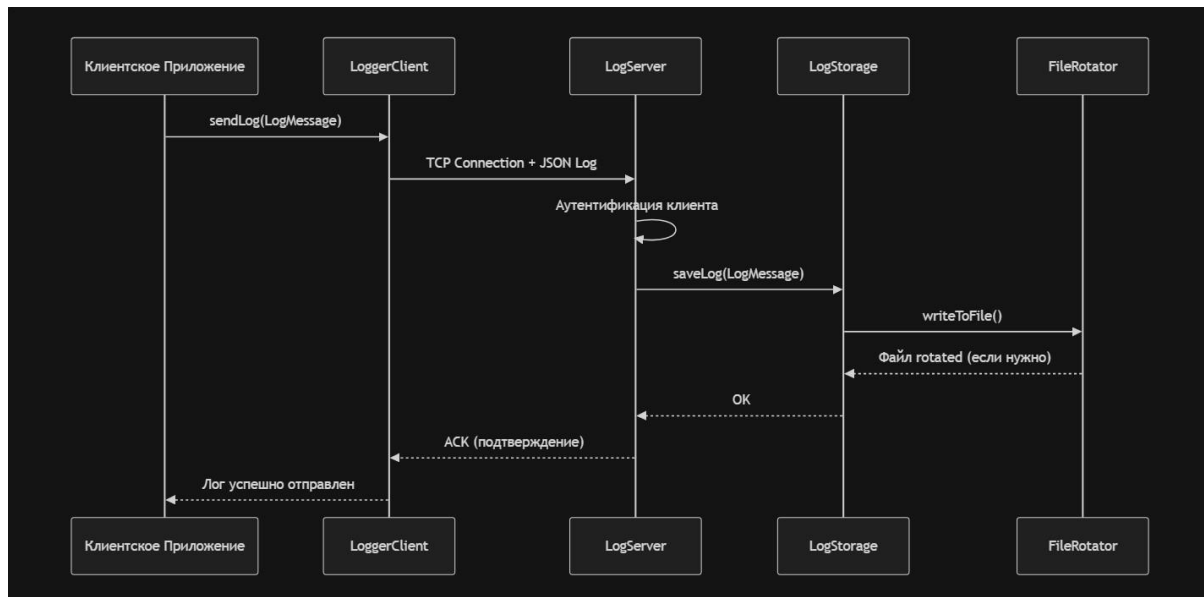


Диаграмма последовательности: Сценарий «Отправка лога клиентом»: Client → LoggerClient → LogServer → LogStorage → FileRotator.



Дополнительные разделы:

3. Применяемые паттерны проектирования

В проекте Distributed Logging System были использованы следующие паттерны проектирования:

3.1 Singleton (Одиночка)

Где применяется: Класс LogServer

Обоснование: Сервер логирования должен существовать в единственном экземпляре во всём приложении, чтобы избежать конфликтов при записи логов и управлении подключениями.

Реализация:

C++

```

LogServer& LogServer::getInstance() {
    static LogServer instance;    // Ленивая инициализация
    return instance;
}
  
```

Преимущества в проекте:

- Гарантируется единственная точка управления сервером
- Упрощается доступ из разных частей приложения
- Легко реализована глобальная конфигурация (порт, фильтры, live view)
-

3.2 Factory Method / Static Factory

Где применяется: Создание объектов LogMessage

Реализация:

C++

```
client.logInfo("AuthService", "Пользователь вошёл");
client.logError("Database", "Ошибка подключения");
client.logFatal("Payment", "Критическая ошибка");
```

Внутри методов logInfo(), logError() и т.д. создаются объекты LogMessage с нужным уровнем.

3.3 Observer (Наблюдатель) — частичная реализация

Где применяется: Механизм Live View в LogServer

Реализация:

C++

```
void LogServer::broadcastLog(const LogMessage& msg) {
    if (liveViewEnabled_ && shouldShow) {
        std::cout << "[LIVE] SAVED: " << msg.toString() <<
std::endl;
    }
    storage_.save(msg);    // "подписчик" — хранилище
}
```

Преимущества: Разделение ответственности — сервер только рассылает логи, а подписчики (хранилище, консоль, будущие интеграции) сами решают, что с ними делать.

3.4 Facade (Фасад)

Где применяется: Класс LoggerClient

Обоснование: Клиенту не нужно знать детали работы с сетью, JSON, сокетами. Он просто вызывает удобные методы.

Пример:

C++

```
LoggerClient client("127.0.0.1", 8080);
client.connect();
client.logInfo("AuthService", "Успешный вход");
client.logError("DB", "Ошибка соединения");
```

За этим фасадом скрывается работа с TcpConnection, сериализация в JSON и т.д.

3.5 RAII (Resource Acquisition Is Initialization)

Где применяется: Работа с сокетами (TcpConnection), потоками, файлами (FileRotator)

Преимущество: Автоматическое освобождение ресурсов при выходе из области видимости (деструкторы закрывают сокеты, файлы и т.д.).

Вывод по паттернам:

Использование классических паттернов проектирования позволило сделать код:

- Более структурированным и читаемым
- Легко расширяемым
- Соответствующим принципам SOLID (особенно Single Responsibility и Dependency Inversion)

4 Предположения и зависимости

- Клиенты используют TCP-соединение.
- Логи передаются в формате JSON.
- Сервер работает под Linux/Windows.

5 Риски

- Потеря логов при разрыве соединения.
- Высокая нагрузка на сервер.
- Проблемы с безопасностью.

6 Глоссарий

- Лог — запись о событии в приложении.
- Ротация — создание новых файлов логов при достижении лимита.
- Observability — наблюдаемость системы.